

Manual de Operações e Versionamento: Lastro App

Este manual técnico estabelece o Ciclo de Vida de Desenvolvimento de Software (SDLC) oficial para o ecossistema financeiro Lastro. O objetivo é garantir a integridade estrutural, rastreabilidade e segurança dos dados entre os ambientes locais, de homologação e de produção.

Fase 1: Preparação Diária (Início do Expediente)

Antes de iniciar o desenvolvimento, o ambiente local deve ser sincronizado com a fonte da verdade no repositório.

1. **Inicialização do Motor:** Certifique-se de que o Docker Desktop está em execução no sistema.
2. **Sincronização de Código:** Abra o VS Code em C:\SIGMA\Lastro-app e atualize a base de código local.

```
git pull origin main
```
3. **Subida da Infraestrutura:** Levante o banco de dados local isolado.

```
npx supabase start
```
4. **Alinhamento do Front-end:** No FlutterFlow, garanta que o ambiente ativo de compilação esteja configurado para *Staging*.

Fase 2: Rotina de Desenvolvimento Híbrido

O desenvolvimento é estritamente segregado entre a manipulação de dados (backend) e a interface gráfica (frontend).

2.1 Manipulação do Banco de Dados (Supabase)

- Realize as alterações estruturais via GUI no Supabase Local (<http://127.0.0.1:54323>).
- Gere o snapshot da infraestrutura (DDL) imediatamente via terminal:

```
npx supabase db diff -f nome_da_alteracao_estrutural
```
- Propague o schema atualizado para o ambiente de Homologação na nuvem (lastro-homologacao):

```
npx supabase db push --db-url "URL_DO_LASTRO_HOMOLOGACAO"
```

2.2 Manipulação de Interface (FlutterFlow)

- Construa telas, conecte componentes, crie consultas e gerencie estados globais.
- Atualize o mapa estrutural no FlutterFlow clicando em **Get Schema** na aba de integração do Supabase.

Fase 3: Consolidação e Versionamento (Fim do Dia)

Todas as alterações sistêmicas devem ser rastreadas e salvas de forma coesa e documentada.

1. **Download do Front-end:** Baixe o código fonte pelo FlutterFlow Desktop e extraia os artefatos substituindo os arquivos em C:\SIGMA\lastro-app\lastro-sigma-sistemas.
2. **Staging de Alterações:** Adicione os novos arquivos (migrações .sql e código Dart) ao palco do Git.

```
git add .
```
3. **Commit Semântico:** Registre as modificações usando prefixos padronizados de engenharia (feat, fix, chore).

```
git commit -m "feat: descricao detalhada das alteracoes efetuadas no periodo"
```
4. **Push e Encerramento:** Envie o pacote consolidado e desligue o motor do banco de dados local.

```
git push origin main  
npx supabase stop
```

Fase 4: Deploy para Produção (Lançamento)

Esta fase deve ser executada exclusivamente durante as janelas de lançamento programadas, após validação rigorosa das operações em ambiente de homologação.

1. Aplique as migrações estruturais diretamente no banco oficial (lastro-financa):

```
npx supabase db push --db-url "URL_DO_LASTRO_FINANCA"
```
2. No FlutterFlow, alterne a injeção de ambiente de desenvolvimento para **Produção**.
3. Execute a compilação do binário final de distribuição (Web, APK ou AAB).

Anexo A: Referência Operacional CLI

Comando	Propósito Técnico
<code>npx supabase start</code>	Inicializa os containeres subjacentes de banco de dados e APIs no ambiente local.
<code>npx supabase stop</code>	Encerra a execução e libera as portas e os recursos alocados.
<code>npx supabase db diff -f [id]</code>	Lê as modificações executadas no Studio UI e as converte em arquivos de migração DDL.
<code>npx supabase db push --db-url</code>	Aplica todos os arquivos de migração rastreados no cluster de banco de dados remoto especificado.